

# **TITANIC SURVIVAL PREDICTION USING MACHINE LEARNING CLASSIFICATION MODELS**

**BY**

**M.VAMSI KRISHNA - 2473A39087**

**SHAIK AFTAB AHMED - 2473A39116**

## **TABLE OF CONTENTS :**

### **TITANIC SURVIVAL PREDICTION USING MACHINE LEARNING CLASSIFICATION MODELS**

#### **1.ABSTRACT**

#### **2.TOOLS AND ENVIRONMENT SETUP**

2.1. CREATED A FOLDER FOR MINI PROJECT :

2.2. CREATED A VIRTUAL ENVIRONMENT FOR THAT MINI PROJECT :

2.3 ACTIVATED THE VIRTUAL ENVIRONMENT

2.4. INSTALLED REQUIRED LIBRARIES IN THE VIRTUAL ENVIRONMENT :

2.5. TITANIC DATASET SETUP FROM KAGGLE USING LINUX TERMINAL

2.6. VIEWING CSV FILES IN PROPER TABLE FORMAT IN VS CODE

#### **3. PROJECT WORKFLOW OVERVIEW**

PHASE 1: PROBLEM UNDERSTANDING

PHASE 2: DATA ACQUISITION & EXPLORATION

PHASE 3: DATA PREPROCESSING

PHASE 4: FEATURE SELECTION & DATA PREPARATION

PHASE 5: MODEL BUILDING

PHASE 6: MODEL EVALUATION

PHASE 7: FINAL MODEL USAGE

#### **4. IMPLEMENTATION PHASE :**

4.1 IMPORTING THE REQUIRED LIBRARIES :

4.2 LOADING THE TRAINING DATASET :

4.3 VERIFYING THE LOADED TRAINING DATASET :

4.5 EXPLORATORY DATA ANALYSIS :

4.6 DATA PREPROCESSING ON TRAINING DATA

4.7 FEATURE SELECTION -DATA PREPARATION

4.8 MODEL BUILDING

4.9 EVALUATION METRICS ( USING VALIDATION DATA )

4.10 CHOOSING BEST MODEL

4.11 LOADING TEST DATASET:

4.12 VERIFYING THE LOADED TEST DATASET

4.13 DATA PREPROCESSING ON TEST DATA

4.14 FEATURE SELECTION

4.15 PREDICTION:

4.16 TEST WITH CUSTOM INPUT

# 1.ABSTRACT

The sinking of the RMS Titanic remains one of the most tragic maritime disasters in history. This project aims to predict passenger survival using supervised machine learning techniques. The problem is formulated as a **binary classification task**, where the target variable Survived indicates whether a passenger survived (1) or not (0).

The dataset used in this project is obtained from the **Titanic - Machine Learning from Disaster** competition hosted on **Kaggle**. It includes passenger details such as ticket class (Pclass), gender (Sex), age, fare, number of siblings/spouses aboard (SibSp), number of parents/children aboard (Parch), and port of embarkation (Embarked).

**Data Preprocessing** is performed to ensure quality and consistency. Missing values in features like Age and Embarked are handled appropriately. Irrelevant columns such as Name, Ticket, and Cabin are removed where necessary. Categorical variables are encoded into numerical format to make them suitable for model training. Feature engineering techniques such as creating Family Size are applied to improve predictive performance.

Three classification algorithms are implemented: **Logistic Regression**, **Decision Tree**, and **Random Forest**. Logistic Regression serves as a baseline linear model to estimate survival probability. Decision Tree identifies survival patterns using rule-based splits. Random Forest, an ensemble learning method, combines multiple decision trees to improve accuracy and reduce over fitting.

Model performance is evaluated using metrics such as **Accuracy**, **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**. Comparative analysis is conducted to determine the most effective model. Among the implemented algorithms, Random Forest generally provides better performance due to its ensemble nature.

This project demonstrates how machine learning techniques can be applied to historical datasets to extract meaningful insights and build predictive systems for classification problems.

## 2.TOOLS AND ENVIRONMENT SETUP

### 2.1. CREATED A FOLDER FOR MINI PROJECT :

**Command** : `mkdir mlminiproject2-2`

### 2.2. CREATED A VIRTUAL ENVIRONMENT FOR THAT MINI PROJECT :

**Command** : `python3 -m venv venv_name`

**Explanation :**

- `python3`
  - Refers to the **Python version 3 interpreter** installed on the system.
  - Ensures that the virtual environment is created using **Python 3**, not Python 2.
- `-m`
  - Stands for **module**.
  - Tells Python to **run a library module as a script** instead of a file.
- `venv`
  - A **built-in Python module** used to create **virtual environments**.
  - A virtual environment is an **isolated workspace** for Python projects.
- `venv_name`
  - The **name of the virtual environment directory**.
  - This folder will contain:
    - A separate Python interpreter
    - Installed libraries
    - Configuration files

**Purpose:**

- Creates an **isolated Python environment** for the project.
- Prevents conflicts between project-specific libraries and system-wide Python packages.
- Ensures **consistent behavior across different systems**.

### 2.3 ACTIVATED THE VIRTUAL ENVIRONMENT

**command** : `source venv_name/bin/activate`

**Explanation :**

- source
  - A shell command used to **execute a script in the current terminal session**.
  - Ensures environment variables are applied immediately.
- venv\_name/
  - Refers to the virtual environment directory created earlier.
- bin/
  - Contains **executable files** related to the virtual environment.
  - Includes Python interpreter and activation scripts.
- activate
  - A shell script that:
    - Modifies the terminal PATH
    - Points Python and pip to the virtual environment
    - Displays the environment name in the prompt

#### **Effect After Activation:**

- Terminal prompt changes to:(venv\_name)
- All Python command and library installations are now **limited to this environment**.
- System Python remains untouched.

## **2.4. INSTALLED REQUIRED LIBRARIES IN THE VIRTUAL ENVIRONMENT :**

**Command : pip install numpy pandas matplotlib scikit-learn**

#### **Libraries Installed:**

numpy  
pandas  
matplotlib  
scikit-learn

#### **Explanation of Each Library:**

##### **NumPy**

- Provides support for:
  - Multi-dimensional arrays
  - Mathematical and numerical operations

Used for:

- Fast computations
- Matrix and vector operations
- Scientific calculations

## **Pandas**

- Used for:
  - Data manipulation and analysis
  - Handling datasets (CSV, Excel, etc.)
- Provides:
  - DataFrames and Series
  - Data cleaning and preprocessing tools

## **Matplotlib**

- A plotting and visualization library.
- Used for:
  - Creating graphs and charts
  - Visualizing trends and patterns in data
- Common plots:
  - Line plots
  - Bar charts
  - Histograms

## **Scikit-learn**

- A machine learning library.
- Used for:
  - Classification
  - Regression
  - Clustering
  - Model evaluation
- Provides ready-to-use ML algorithms and utilities.

## **Python and pip**

**Python** is a high-level programming language used to write and execute machine learning code. **pip** is the package manager for Python. It is used to install, update, and remove external libraries that extend Python's functionality

## 2.5. TITANIC DATASET SETUP FROM KAGGLE USING LINUX TERMINAL

This section documents the **complete and clean workflow** of downloading the Titanic dataset from Kaggle, moving it into a project directory, extracting it, and safely managing the ZIP file using Linux terminal command.

### 1. Navigating to Kaggle and Downloading the Dataset

1. Open a web browser and go to **Kaggle** (<https://www.kaggle.com>).
2. Log in using your Kaggle account.
3. In the search bar, type "**Titanic dataset**".
4. Open the dataset commonly named "**Titanic - Machine Learning from Disaster**".
5. Click on the **Download** button.

Kaggle downloads datasets as **ZIP files** by default.

After downloading, the ZIP file (archive.zip) will be stored in the system's **Downloads** directory.

---

### 2. Understanding the Downloads Directory

In Linux, each user has a home directory represented by the symbol ~.

- ~/Downloads → Refers to the **Downloads folder** of the current user.

You can verify the downloaded file using:

```
ls ~/Downloads
```

---

### 3. Moving the ZIP File to the Project Folder

The dataset ZIP file is moved from the Downloads folder to the project directory named MLMINIPROJECT2-2 using the mv command.

#### command Used

```
mv ~/Downloads/archive.zip ~/MLMINIPROJECT2-2
```

#### Explanation of Each Term

- mv → Move command (used to move or rename files)
- ~/Downloads/archive.zip → Source file path (ZIP file in Downloads)
- ~/MLMINIPROJECT2-2 → Destination project directory

After this command, the ZIP file is no longer in Downloads and is now inside the project folder.

---

#### 4. Navigating to the Project Directory

Before extracting the ZIP file, we navigate into the project folder.

```
cd ~/MLMINIPROJECT2-2
```

##### Explanation

- `cd` → Change directory
- `~/MLMINIPROJECT2-2` → Path of the ML project directory

You can confirm the file presence using:

```
ls
```

---

#### 5. Extracting the ZIP File

The dataset is extracted using the `unzip` command.

```
unzip archive.zip
```

##### Explanation

- `unzip` → command used to extract ZIP files
- `archive.zip` → ZIP file containing the Titanic dataset

After extraction, dataset files such as CSV files become available for use in the project.

---

#### 6. Removing the ZIP File After Extraction

Once the contents are successfully extracted, the ZIP file is no longer required. It is removed using the `rm` command.

```
rm archive.zip
```

##### Important Note on `rm`

- `rm` permanently deletes files
- Files deleted using `rm` **do not go to Trash**
- There is **no undo** operation

This step helps keep the project directory clean and organized.



## **2.6. VIEWING CSV FILES IN PROPER TABLE FORMAT IN VS CODE**

Visual Studio Code opens CSV files as plain text by default. This is expected behavior because a CSV file is a text file. To make the data readable in a table-like format, we use a VS Code extension ( Rainbow csv ) .

### **Step 1 : Ensure the File Extension is CSV**

If the file has a different extension, rename it to .csv and reopen it in VS Code. Rainbow CSV command will not appear if the file is not recognized as a CSV file.

### **Step 2 Set the Language Mode to CSV**

Look at the bottom-right corner of the VS Code window.  
If it shows Plain Text, click on it and select CSV.  
This tells VS Code to treat the file as a CSV file.

### **Step 3 Align the CSV Data Using Rainbow CSV**

Open the command Palette using Ctrl + Shift + P.  
Type Rainbow CSV: Align and execute the command.  
The comma-separated values will align into columns, making the file look like a table.

## 3. PROJECT WORKFLOW OVERVIEW

### PHASE 1: PROBLEM UNDERSTANDING

#### Step 1: Define the Problem

- Identify the task as **Supervised Learning – Binary Classification**.
  - Objective: Predict whether a passenger survived the Titanic disaster.
  - Target Variable: Survived
    - 0 → Not Survived
    - 1 → Survived
- 

### PHASE 2: DATA ACQUISITION & EXPLORATION

#### Step 2: Load Dataset

- Import Titanic dataset into a DataFrame.
- Verify number of samples and features.

#### Step 3: Understand Dataset Structure

- Identify numerical and categorical features.
- Confirm target variable (Survived).
- Inspect column names, data types, and sample records.

#### Step 4: Exploratory Data Analysis (EDA)

- Analyze survival distribution.
  - Study impact of key features (Sex, Pclass, Age, Fare).
  - Detect missing values, outliers, and irrelevant columns.
- 

### PHASE 3: DATA PREPROCESSING

#### Step 5: Handle Missing Values

- Apply Mean/Median for numerical features.
- Apply Mode for categorical features.

#### Step 6: Remove Irrelevant Features

- Drop non-predictive identifiers (PassengerId, Name, Ticket).
- Retain meaningful attributes.

#### Step 7: Encode Categorical Variables

- Convert categorical features into numerical format.
  - Sex → Binary Encoding
  - Embarked → One-Hot/Label Encoding

#### **Step 8: Feature Scaling**

- Normalize or standardize numerical features (Age, Fare) where required.
- 

### **PHASE 4: FEATURE SELECTION & DATA PREPARATION**

#### **Step 9: Feature Selection**

- Select relevant features using correlation analysis and domain knowledge.
- Remove weak or redundant variables.

#### **Step 10: Train-Test Split**

- Divide dataset into Training and Testing sets (70:30 or 80:20).
  - Ensure no data leakage.
- 

### **PHASE 5: MODEL BUILDING**

#### **Step 11: Algorithm Selection**

- Implement classification models:
  - Logistic Regression
  - Decision Tree
  - Random Forest
- Select models based on interpretability and performance.

#### **Step 12: Model Training**

- Train models using the training dataset.
  - Learn patterns between features and survival outcome.
- 

### **PHASE 6: MODEL EVALUATION**

#### **Step 13: Prediction**

- Generate survival predictions on test data.

#### **Step 14: Performance Evaluation**

- Evaluate using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-Score

### **Step 15: Model Optimization**

- Improve performance through hyperparameter tuning and model comparison.
- 

## **PHASE 7: FINAL MODEL USAGE**

### **Step 16: Custom Prediction**

- Accept new passenger input.
- Apply same preprocessing pipeline.
- Predict survival outcome.

### **Step 17: Conclusion**

- Summarize model performance , Identify key survival factors.
- Discuss limitations and future improvements.

## **4. IMPLEMENTATION PHASE :**

### **4.1 IMPORTING THE REQUIRED LIBRARIES :**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
```

#### **import :**

A Python keyword used to bring an external library or module into your program so you can use its functions, classes, and tools.

#### **from :**

A Python keyword used to import a specific part (module, class, or function) from a larger library instead of importing the entire library.

#### **as :**

A Python keyword used to give a short alias (nickname) to a library or module for easier usage.

#### **. (dot operator) :**

Used to access something inside a library or module.

Example: sklearn.preprocessing

Here, preprocessing is inside sklearn.

#### **library :**

A collection of pre-written code created by others to perform specific tasks.

#### **module :**

A single Python file inside a library that contains related functions or classes.

#### **package :**

A collection of modules grouped together inside a library.

#### **framework :**

A larger structured library that provides a complete system to build applications (e.g., machine learning systems).

**sklearn :**

Short name for Scikit-learn.

It is a Machine Learning library in Python used for building, training, and evaluating ML models.

**preprocessing :**

A module inside sklearn used to clean, scale, or transform data before training a model.

**linear\_model :**

A module inside sklearn that contains linear algorithms like Logistic Regression.

**tree :**

A module inside sklearn that contains decision tree algorithms.

**ensemble :**

A module inside sklearn that contains models which combine multiple algorithms together to improve performance.

**model\_selection :**

A module used to split data and select best models.

**metrics :**

A module used to measure how well a machine learning model performs.

---

**1. import pandas as pd**

pandas :

A Python library used for data manipulation and analysis.

It works mainly with tables (rows and columns).

pd :

Alias name given to pandas.

Data structure used in pandas:

DataFrame :

A 2D table structure (like Excel sheet).

Series :

A single column of data.

---

**2. import matplotlib.pyplot as plt**

matplotlib :

A Python library used for data visualization (drawing graphs).

pyplot :

A module inside matplotlib used to create plots easily.

plt :

Alias for pyplot.

Used for:

Line graphs, bar charts, histograms, scatter plots.

---

### **3. import seaborn as sns**

seaborn :

A Python visualization library built on top of matplotlib.  
It gives more attractive and statistical graphs.

sns :

Alias for seaborn.

Used for:

Heatmaps, distribution plots, box plots, correlation maps.

---

### **4. import numpy as np**

numpy :

A Python library used for numerical operations and mathematical calculations.

np :

Alias for numpy.

Core structure:

ndarray :

A fast multi-dimensional array for numbers.

Used for:

Matrix operations, linear algebra, mathematical functions.

---

### **5. from sklearn.preprocessing import StandardScaler**

StandardScaler :

A class used to standardize data.

Why scaling needed:

Machine learning models perform better when features are on same scale.

Formula used:

$Z = (X - \text{mean}) / \text{standard deviation}$

Result:

Data will have mean = 0

Standard deviation = 1

---

### **6. from sklearn.linear\_model import LogisticRegression**

LogisticRegression :

A classification algorithm used for binary classification problems.

It predicts probability using Sigmoid function.

Formula:

$$z = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Sigmoid function:

$$P = 1 / (1 + e^{(-z)})$$

Output:

Value between 0 and 1 (probability).

---

## **7. from sklearn.tree import DecisionTreeClassifier**

DecisionTreeClassifier :

A classification model that makes decisions based on conditions (if-else structure).

It splits data into branches based on features.

Split criteria formulas:

Gini Index:

$$\text{Gini} = 1 - \sum (p_i)^2$$

Entropy:

$$\text{Entropy} = - \sum (p_i \log_2 p_i)$$

Where  $p_i$  = probability of class.

---

## **8. from sklearn.ensemble import RandomForestClassifier**

RandomForestClassifier :

An ensemble learning model that combines multiple decision trees.

ensemble :

Means combining multiple models to improve accuracy.

How it works:

1. Builds many decision trees
2. Each tree gives prediction
3. Final output = majority vote

Improves:

Accuracy

Reduces overfitting

---

## **9. from sklearn.model\_selection import train\_test\_split**

train\_test\_split :

A function used to divide dataset into:

Training data

Testing data



Purpose:  
Train model on one part  
Test model on unseen data

Common split:  
80% training  
20% testing

---

#### **10.        `from sklearn.metrics import accuracy_score`**

`accuracy_score` :  
Measures how many predictions are correct.

Formula:

$\text{Accuracy} = (\text{Correct Predictions}) / (\text{Total Predictions})$

---

#### **11. `from sklearn.metrics import confusion_matrix`**

`confusion_matrix` :  
A table used to evaluate classification performance.

Structure:

True Positive (TP)  
True Negative (TN)  
False Positive (FP)  
False Negative (FN)

---

#### **12.        `from sklearn.metrics import precision_score`**

`precision_score` :  
Measures how many predicted positives are actually correct.

Formula:

$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

---

#### **13.        `from sklearn.metrics import recall_score`**

`recall_score` :  
Measures how many actual positives are correctly predicted.

Formula:

$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

---

#### **14.        `from sklearn.metrics import f1_score`**

f1\_score :  
Harmonic mean of precision and recall.

Formula:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

---

## 15. `from sklearn.metrics import classification_report`

classification\_report :  
Generates a complete summary including:

Precision  
Recall  
F1 Score  
Support (number of samples per class)

## 4.2 LOADING THE TRAINING DATASET :

→ `df = pd.read_csv("./titanic/train.csv")`

This line is used to read a CSV (Comma Separated Values) file and load the dataset into a pandas DataFrame. Here, `pd` refers to the pandas library which is used for data analysis in Python. The function `read_csv()` is used to read data from the specified file path. The path `"./titanic/train.csv"` indicates that the file named `train.csv` is located inside the `titanic` folder in the current working directory. The variable `df` stores the dataset in a tabular format consisting of rows and columns, which makes it easier to perform data analysis and machine learning operations.

## 4.3 VERIFYING THE LOADED TRAINING DATASET :

### 1. INSPECTING THE COLUMN NAMES :

→ `df.columns`

This is used to display the **column names** present in the DataFrame `df`. In pandas, `columns` is an attribute that returns all the column labels of the dataset in a list-like format. It helps us understand the features or variables available in the dataset for data analysis and machine learning tasks

**output :**

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',  
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'], dtype='str')
```

### **PassengerId**

This is a unique identification number assigned to each passenger in the dataset. It is mainly used for indexing and identification purposes and does not contribute to prediction.

## **Survived**

This indicates whether the passenger survived or not.

0 = Did not survived , 1 = Survived

This is usually the **target variable** in Titanic survival prediction models.

## **Pclass**

Represents the passenger class or ticket class.

- 1 = First class , 2 = Second , class3 = Third class  
It reflects the socioeconomic status of the passenger.

## **Name**

Contains the full name of the passenger. It is a textual feature and is usually not directly used for prediction but can be used for feature extraction.

## **Sex**

Indicates the gender of the passenger (male or female). This is an important feature in survival analysis.

## **Age**

Represents the age of the passenger. It is a numerical feature and is useful in analyzing survival probability based on age groups.

## **SibSp**

Represents the number of siblings or spouses the passenger had aboard the Titanic. It helps analyze family relationships during survival prediction.

## **Parch**

Represents the number of parents or children the passenger had aboard. It is used to understand family size influence on survival chances.

## **Ticket**

Represents the ticket number of the passenger. It is mostly an identifier and is usually not directly useful for prediction.

## **Fare**

Represents the ticket fare paid by the passenger. It is a numerical feature that may indicate passenger class and wealth.

## **Cabin**

Indicates the cabin number assigned to the passenger. This column has many missing values but can sometimes be used for feature engineering.

## **Embarked**

Represents the port where the passenger boarded the Titanic.

C = Cherbourg , Q = Queenstown , S = Southampton

**dtype = str** means that the **data type of the column names (index labels)** is string (text format).

## 2. INSPECTING SAMPLE RECORDS :

→ **df.head()**

|   | PassengerId | Survived | Pclass | Name                                              | Sex    | Age  | SibSp | Parch | Ticket           | Fare    | Cabin | Embarked |
|---|-------------|----------|--------|---------------------------------------------------|--------|------|-------|-------|------------------|---------|-------|----------|
| 0 | 1           | 0        | 3      | Braund, Mr. Owen Harris                           | male   | 22.0 | 1     | 0     | A/5 21171        | 7.2500  | NaN   | S        |
| 1 | 2           | 1        | 1      | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1     | 0     | PC 17599         | 71.2833 | C85   | C        |
| 2 | 3           | 1        | 3      | Heikkinen, Miss. Laina                            | female | 26.0 | 0     | 0     | STON/O2. 3101282 | 7.9250  | NaN   | S        |
| 3 | 4           | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)      | female | 35.0 | 1     | 0     | 113803           | 53.1000 | C123  | S        |
| 4 | 5           | 0        | 3      | Allen, Mr. William Henry                          | male   | 35.0 | 0     | 0     | 373450           | 8.0500  | NaN   | S        |

## 3. INSPECTING TOTAL NUMBER OF RECORDS AND COLUMNS :

→ **print(len(df))**

This line is used to print the **number of rows (records)** present in the DataFrame df.  
In simple terms, it shows **how many passengers' data entries are there in the dataset.**

Output : 891

---

→ **print(len(df.columns))**

This line is used to print the **number of columns (features/attributes)** present in the DataFrame df.  
In simple terms, it shows **how many variables or features are available in the dataset.**

Output : 12

## 4.4 UNDERSTANDING THE DATA STRUCTURE :

### 1. INSPECTING THE DATA TYPES OF EACH COLUMN :

→ **df.dtypes**

Display the **data types of each column** in the DataFrame df.

In pandas, each column can have a different data type such as:

- **int64** → Integer numbers
- **float64** → Decimal numbers
- **object** → Text or mixed data (usually strings)
- **bool** → True/False values

**output :**

```

PassengerId    int64
Survived       int64
Pclass         int64
Name           str
Sex            str
Age            float64
SibSp          int64
Parch          int64
Ticket         str
Fare           float64
Cabin          str
Embarked       str
dtype: object

```

## 2. IDENTIFYING THE NUMERICAL FEATURES :

→ **numerical\_features=df.select\_dtypes(include=["int64","float64"]).columns.tolist()**

selects all columns from dataframe df that have **numerical data types** such as:

- Integer values (int64)
- Decimal values (float64)

Then converts the column names into a **Python list**.

→ **if target in numerical\_features :**  
**numerical\_features.remove(target)**

checks whether the **target column** is present in the numerical features list.

If yes, it removes the target column because:

- Target variable should not be used as an input feature during model training.

→ **print(numerical\_features)**

**output :**

```
['PassengerId', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
```

## 3. IDENTIFYING CATEGORICAL FEATURES :

→ **cateogorical\_features = df.select\_dtypes(include="string").columns.tolist()**

**df.select\_dtypes(include = “ string “ )**

Selects all columns in dataframe df that contain **string/text data**.

These are usually **categorical variables** like:

- Names , Categories , Labels , Gender

**.columns.tolist() :**

Converts the selected column names into a **Python list format** so you can easily use them in further processing like encoding or visualization.

→ **print( categorical\_features )**

**output :**

['Name', 'Sex', 'Ticket', 'Cabin', 'Embarked']

## **4.5 EXPLORATORY DATA ANALYSIS :**

### **1. SURVIVAL DISTRIBUTION :**

→ **plt.figure(figsize=(6, 4))**

Creates a new plot figure with size **6 inches (width) × 4 inches (height)**.

This controls the **visual size** of the graph.

→ **sns.countplot(data=df,x="Survived")**

Creates a **count plot** using Seaborn.

It counts how many records are present in each category of the column survived.

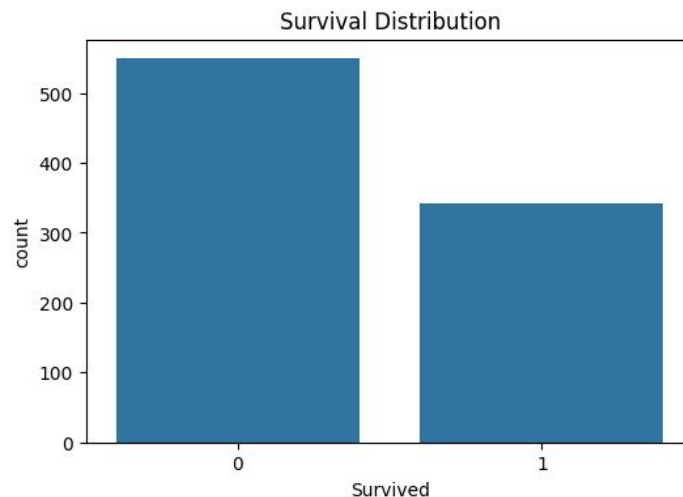
In the Titanic dataset context : 0 → Not survived , 1 → Survived

So this plot shows **how many passengers survived vs not survived**.

→ **plt.title("Survival Distribution")**

Adds a title to the plot: **"Survival Distribution"**.

→ **plt.show()**



### Core Data Observations

- **Majority Non-Survival:** More people in this dataset did not survive (0) than those who did (1).
- **Exact Counts:**
  - **Survived (0):** Approximately **550** individuals.
  - **Survived (1):** Approximately **340** individuals.
- **Total Sample Size:** The total population represented in this chart is roughly **890** individuals (consistent with the famous Titanic dataset).

### Key Inferences

- **Survival Rate:** Roughly **38%** of the individuals survived, while **62%** did not.
- **Class Imbalance:** There is a noticeable "imbalance" in the classes. If you are using this for machine learning, a model that simply guesses "0" every time would be accurate about 62% of the time.
- **High Mortality:** The death toll (0) is nearly **1.6 times higher** than the number of survivors (1).

→ `print("Survival counts :",df['Survived'].value_counts())`

output :

```
Survival counts : Survived
0    549
1    342
Name: count, dtype: int64
```

→ `print("Survival % :",df['Survived'].value_counts(normalize=True)*100)`

output :

```
Survival percentage : Survived
0    61.616162
```

1 38.383838

Name: proportion, dtype: float64

`value_counts(normalize=True)` → Counts the occurrences of each unique value (0 or 1) and normalizes it, giving the **proportion instead of raw counts**. \*100 Converts the proportion into percentage.

## 2. FEATURE WISE SURVIVAL ANALYSIS :

### 1. SURVIVAL BY CATEGORICAL FEATURES :

```
→ categorical_features_main = ['Sex', 'Embarked']
for col in categorical_features_main :
    plt.figure(figsize=(6,4))
    sns.countplot(data= df ,x = col ,hue = 'Survived')
    plt.title(f"Survival distribution by {col}")
    plt.show()
```

#### 1. categorical\_features\_main = ['Sex', 'Embarked']

- Creates a list of categorical columns.
- Here, **Sex (Gender)** and **Embarked (Boarding port)** are selected for analysis.

#### 2. for col in categorical\_features\_main:

- Loop is used to iterate through each categorical feature one by one.

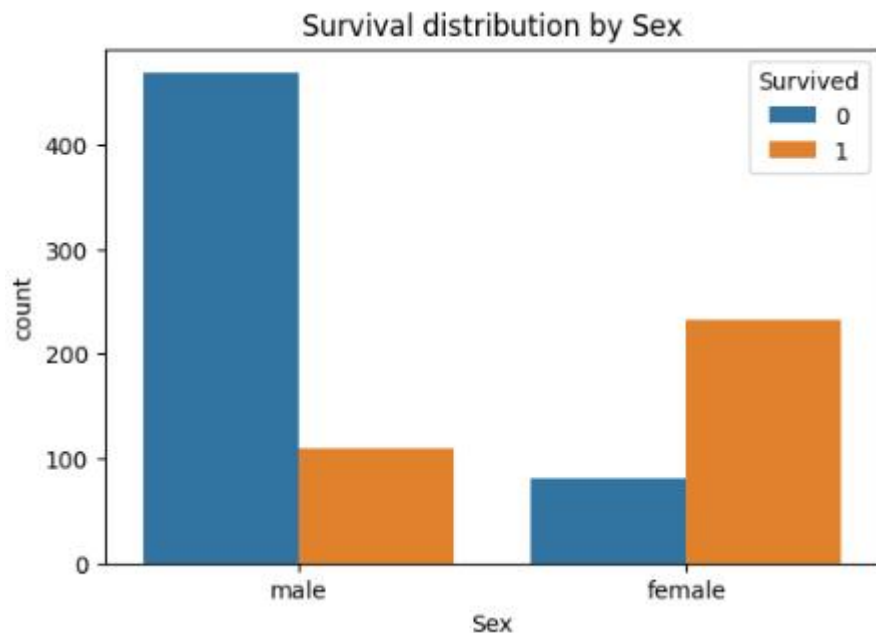
#### 3. sns.countplot(data=df, x=col, hue='Survived')

- Creates a count plot showing the number of passengers.
- `x=col` → Shows categories like male/female or boarding ports.
- `hue='Survived'` → Separates data into survived and not survived groups.

#### 4. plt.title(f"Survival distribution by {col}")

- Sets title dynamically based on column name.



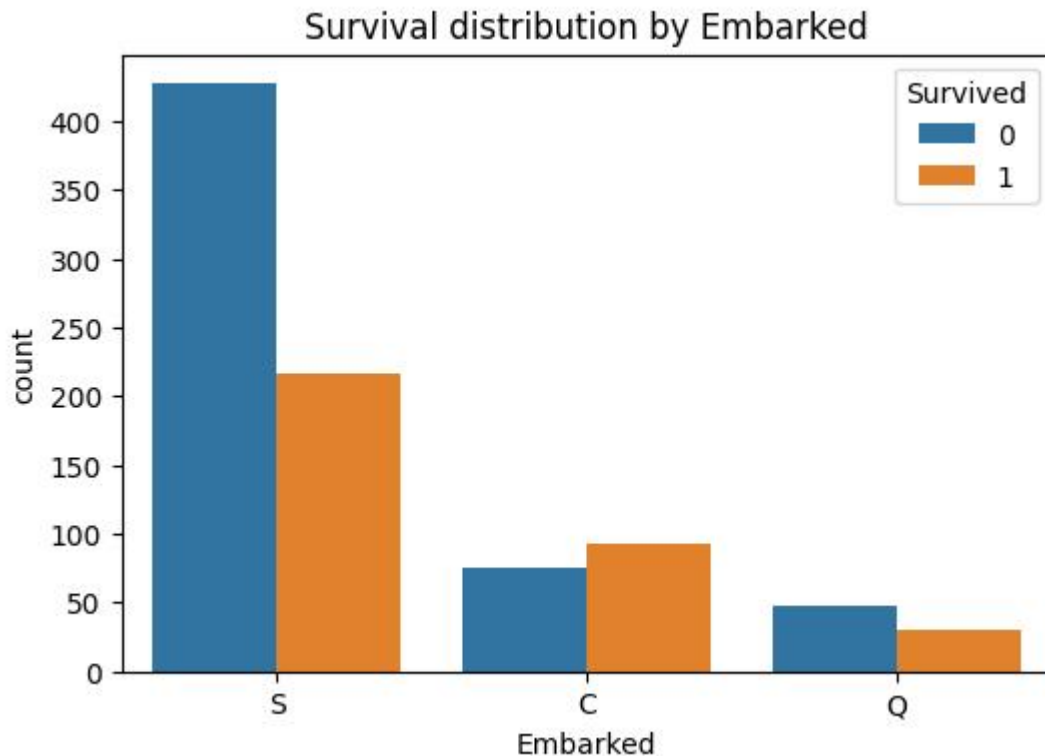


### Key Observations

- **High Male Mortality:** The vast majority of males in this dataset did not survive (0).
- **High Female Survival:** The majority of females in this dataset survived (1).
- **Disproportionate Representation:** There were significantly more males on board (roughly 580 total) than females (roughly 310 total).

### Detailed Inferences

- **Survival Disparity:** While there were fewer females overall, their survival count (roughly 230) is more than double the survival count of males (roughly 110).
- **Mortality Rates:**
  - **Males:** Roughly 80% of the male population did not survive.
  - **Females:** Roughly 75% of the female population survived.
- **Predictive Strength:** Gender is a very strong predictor of survival in this dataset. If you knew only the sex of a passenger, guessing "Female = Survived" and "Male = Deceased" would yield a high degree of accuracy.



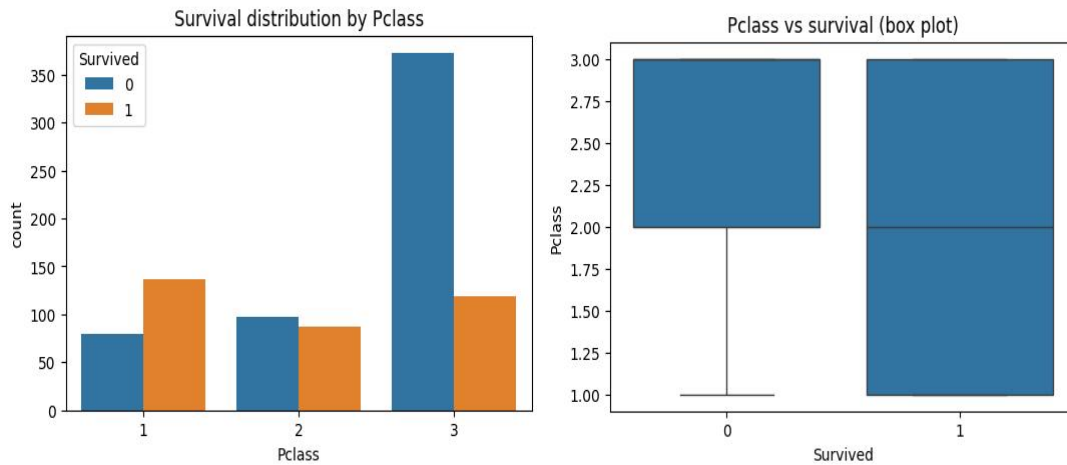
### Survival by Port of Embarkation

- **Dominance of Port S:** The vast majority of passengers embarked at **Port S (Southampton)**.
- **Highest Mortality at Port S:** Port S shows the highest number of non-survivors (over **425**) and the lowest survival rate among the three ports.
- **Survival Advantage at Port C:** **Port C (Cherbourg)** is the only embarkation point where the number of survivors (roughly **95**) actually exceeded the number of those who did not survive (roughly **75**).
- **Lowest Volume at Port Q:** **Port Q (Queenstown)** had the fewest passengers overall, with a survival count of approximately **30** and a non-survival count near **50**.

## 2. SURVIVAL BY NUMERICAL FEATURES :

```
→ numerical_features_main = ['Pclass','SibSp','Parch']
for col in numerical_features_main :
    plt.figure(figsize=(6,4))
    sns.countplot(data = df , x =col,hue = 'Survived')
    plt.title(f"Survival distribution by {col}")
    plt.show()
plt.figure(figsize=(6,4))
sns.boxplot(data = df ,x='Survived',y=col)
```

```
plt.title(f'{col} vs survival (box plot)')
plt.show()
```



### 1. The "Survival Advantage" of First Class

The bar chart clearly shows that **Pclass 1** is the only category where survivors (orange) significantly outnumber those who did not survive (blue).

- **Inference:** If you were in First Class, your probability of survival was greater than 50%.

### 2. The "Mortality Rate" of Third Class

The bar chart reveals a massive disparity in **Pclass 3**. While it has a survival count comparable to Pclass 1 and 2, the number of deaths is nearly 4x higher than in the other classes.

- **Inference:** Third Class had the highest absolute number of casualties, making it the most "dangerous" class to be in.

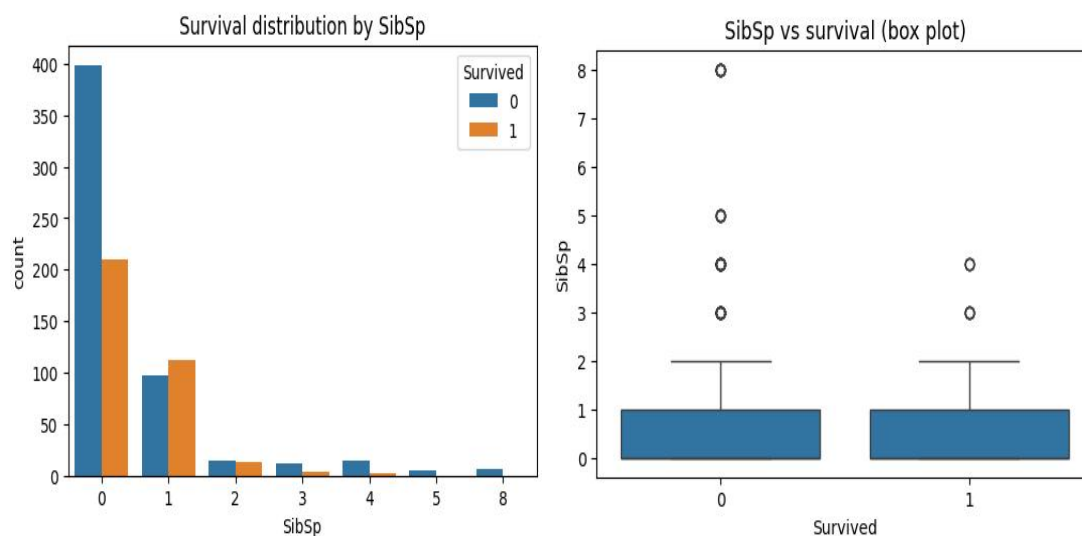
### 3. Median Class of Survivors

- **For Survivors (1):** The median line (the horizontal line inside the box) sits at **2.0**. This indicates a balanced spread of survivors across all classes, but with a center of gravity in the middle class.
- **For Non-Survivors (0):** The median line is pushed up to **3.0**.
- **Inference:** The "typical" person who did not survive was a Third Class passenger, whereas the "typical" survivor came from a higher social standing (First or Second Class).

### 4. Categorical Variance

The box for **Survived = 1** is much larger (covering the full range from 1 to 3) compared to the box for **Survived = 0** (which is concentrated between 2 and 3).

- **Inference:** Survival was relatively well-distributed across all classes (though skewed towards 1), but **death was heavily concentrated in Classes 2 and 3**.



### 1. The "Sweet Spot" for Survival

The bar chart shows that passengers with **exactly 1 sibling or spouse** (SibSp = 1) had the best survival outcome.

- **Inference:** This is the only category where the number of survivors (orange) actually exceeds the number of those who perished (blue). Traveling with one other person appears to have provided a survival advantage.

### 2. Isolation vs. Risk

The vast majority of passengers traveled alone (SibSp = 0).

- **Inference:** While this group had the highest absolute number of survivors, they also faced a significantly higher risk, with nearly double the number of deaths compared to survivors.

### 3. Large Families Faced Extreme Peril

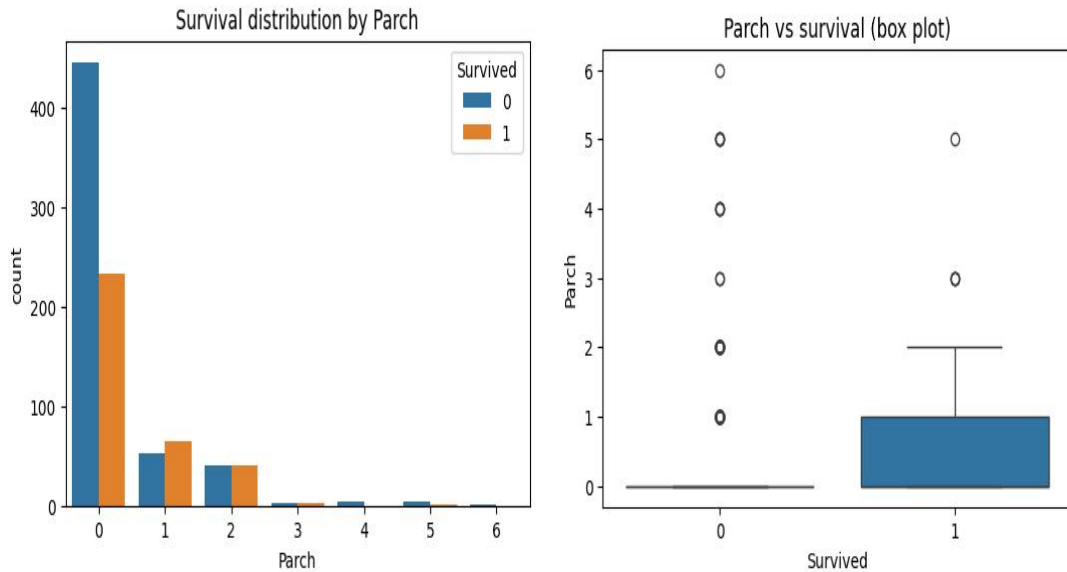
As the number of siblings/spouses increases beyond 2, the orange bars (survivors) nearly disappear.

- **Inference:** For passengers with **SibSp of 5 or 8**, the survival count is zero. Large family units likely found it much harder to coordinate an escape or reach lifeboats together.

### 4. Outlier Analysis

The box plot highlights significant outliers in the data:

- **Non-Survivors (0):** There are extreme outliers at **SibSp = 5 and 8**.
- **Survivors (1):** The maximum SibSp for anyone who survived was **4**.
- **Inference:** A high SibSp count was almost an absolute predictor of non-survival. The "spread" of survivors is much tighter, indicating that those who made it off the ship generally had very few to no family members with them.



### 1. The "Solo Traveler" Penalty

The first chart (Survival distribution) shows that the vast majority of passengers had a **Parch value of 0**.

- While they make up the largest group of survivors in absolute numbers, they also have the **highest mortality rate**.
- The blue bar (died) is nearly double the size of the orange bar (survived) for this group. This suggests that traveling without parents or children significantly decreased one's chances of survival.

### 2. The "Sweet Spot" for Survival

There is a visible shift in the survival ratio for passengers with **Parch 1 or 2**:

- **Parch 1:** This is the only category where the number of survivors (orange) clearly exceeds the number of those who perished (blue).
- **Parch 2:** The survival and mortality rates are nearly equal (50/50).
- **Inference:** Having a small family unit onboard (1–2 parents/children) appears to have been a protective factor, likely due to the "women and children first" evacuation policy, as these passengers were more likely to be part of a family group.

### 3. Large Families Faced Greater Risk

As the Parch count increases to **3, 4, 5, and 6**, the total count of passengers drops significantly, but the survival rate drops as well.

- The box plot shows that higher Parch values (4, 5, and 6) are almost exclusively associated with **non-survival (0)**, represented by the outliers.
- **Inference:** Large families likely struggled to stay together or find enough space in lifeboats during the chaos, leading to lower survival rates compared to small families.

### 3. DETECTING OUTLIERS :

→ for col in numerical\_features\_main:

```
q1 = df[col].quantile(0.25)
q3 = df[col].quantile(0.75)
iqr = q3 - q1
lower = q1 - (1.5) * iqr
upper = q3 + (1.5) * iqr
outliers = df[(df[col]<lower) | (df[col]>upper)]
print(f"{col} Outliers :\n",outliers[col])
```

**output :**

Pclass Outliers :

Series([], Name: Pclass, dtype: int64)

SibSp Outliers :

```
7    3
16   4
24   3
27   3 .....
```

Name: SibSp, dtype: int64

Parch Outliers :

```
7    1
8    2
10   1 .....
```

Name: Parch, Length: 213, dtype: int64

**Pclass**

- No outliers were detected.
- This means Pclass values are well distributed within expected ranges.

**SibSp (Number of siblings/spouses aboard)**

- Several outliers were detected.
- Higher SibSp values (like 3, 4, etc.) are rare compared to the majority.
- This suggests most passengers traveled with **0–1 companions**, while few traveled with larger family groups.

**Parch (Number of parents/children aboard)**

- Many outliers were detected (213 records).

- This indicates that larger family sizes were less common.
- Most passengers traveled with **small family sizes**, while some had unusually large family counts.

#### 4. SKEWNESS CHECK :

```
→ for col in numerical_features_main:
    skewness = df[col].skew()
    print(f"{col} Skewness :{skewness:.2f}")
```

**output :**

```
Pclass Skewness :-0.63
SibSp Skewness :3.70
Parch Skewness :2.75
```

Skewness is used to measure the asymmetry of data distribution. It tells us whether the data is leaning towards the left or right side of the distribution curve.

This function measures how much the data deviates from a **perfectly symmetrical (normal) distribution**.

- **Skewness = 0** → Perfectly symmetrical distribution (ideal bell curve).
- **Skewness > 0** → **Right skewed (positively skewed)**
  - Tail is longer on the right side.
  - Most values are concentrated on the left.
- **Skewness < 0** → **Left skewed (negatively skewed)**
  - Tail is longer on the left side.
  - Most values are concentrated on the right.

#### 5. IRRELEVANT FEATURES :

##### 1. CORRELATION :

```
→ correlation = df.select_dtypes(include=["int64","float64"]).corr()
target_corr = correlation['Survived'].drop('Survived')
print(target_corr)
```

**output :**

```
PassengerId -0.005007
Pclass      -0.338481
Age         -0.077221
SibSp       -0.035322
Parch       0.081629
Fare        0.257307
Name: Survived, dtype: float64
```

Correlation analysis is used to understand **how strongly numerical features are related to each other**, especially how features are related to the **target variable** (in your case: **Survived**).

### Step 1: Select Numerical Features & Compute Correlation

```
correlation = df.select_dtypes(include=["int64","float64"]).corr()
```

- Selects only numerical columns.
- Computes **correlation coefficient** between every pair of numerical variables.

### Step 2: Correlation with Target Variable

```
target_corr = correlation['Survived'].drop('Survived')
```

- Extracts correlation values of all features with the **Survived** column.
- `drop('Survived')` removes self-correlation (which is always 1).

### Correlation Value Interpretation

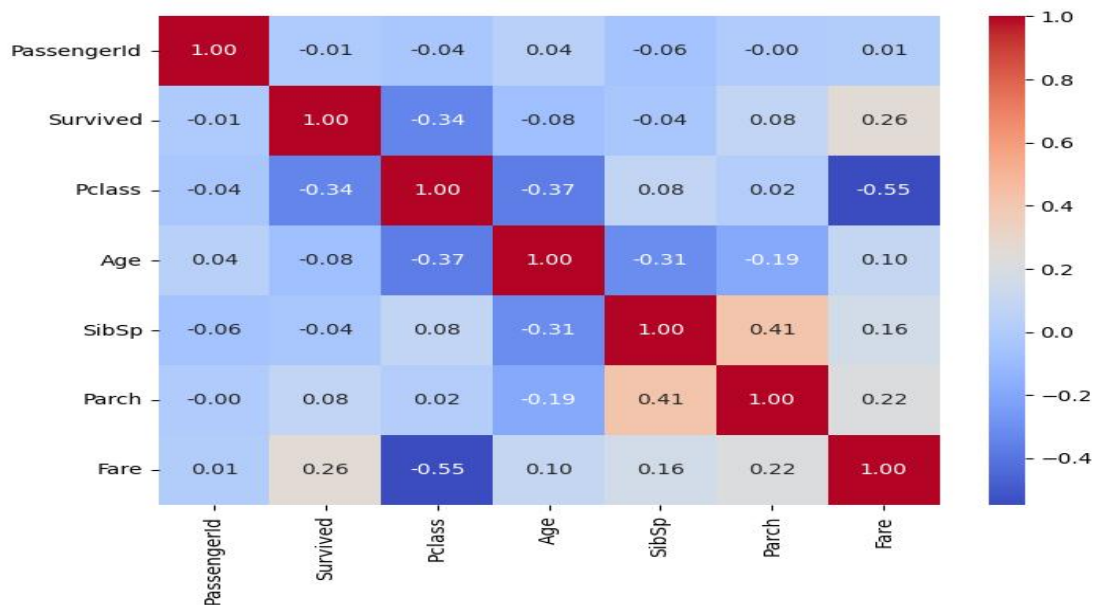
Correlation values range between **-1 and +1**:

- **+1 → Strong positive correlation**
  - As feature value increases → Survival probability increases.
- **0 → No correlation**
  - Feature has little impact on survival.
- **-1 → Strong negative correlation**
  - As feature value increases → Survival probability decreases.

→ 

```
plt.figure(figsize=(8,6))  
sns.heatmap(correlation, annot=True,cmap='coolwarm',fmt='.2f')  
plt.show()
```





## 1. Key Drivers of Survival

Looking at the **Survived** row/column, we can see which factors most influenced whether a passenger lived:

- **Pclass (-0.34):** This is the strongest negative correlation in the survival column. It means as the class number increases (from 1st to 3rd), the chance of survival **decreased**.
- **Fare (0.26):** This is a positive correlation. Higher fares (usually associated with 1st class) led to a **higher** chance of survival.
- **Age (-0.08):** A very weak negative correlation, suggesting that while younger people had a slightly better chance, age alone wasn't as decisive as class or wealth.

## 2. Strongest Relationships in the Data

- **Pclass and Fare (-0.55):** This is the strongest correlation on the entire map (excluding the 1.00 diagonal). It's a strong negative relationship: as the Pclass number goes down (approaching 1st class), the Fare goes up significantly.
- **SibSp and Parch (0.41):** A moderate positive correlation. This makes sense—passengers traveling with siblings or spouses (SibSp) were also likely to be traveling with parents or children (Parch), representing family units.
- **Pclass and Age (-0.37):** This suggests that older passengers were more likely to be in the higher classes (1st class), while younger passengers were more common in 3rd class.

## 3. The "PassengerId" Irrelevance

- **PassengerId (approx 0.00):** You'll notice PassengerId has almost zero correlation with everything else (values like -0.01, -0.04, 0.01).

- **Inference:** This confirms that the ID is just a random label and has no predictive power for survival. You can safely drop this column during machine learning modeling.

```
→ # Temporary mapping for visualization only
df['Sex_encoded'] = df['Sex'].map({'female': 1, 'male': 0})
print(df[['Sex_encoded', 'Survived']].corr())
```

**output :**

```
Sex_encoded Survived
Sex_encoded    1.000000  0.543351
Survived       0.543351  1.000000
```

```
df['Sex_encoded'] = df['Sex'].map({'female': 1, 'male': 0})
```

Converts text values into numbers because machine learning and correlation calculations require numeric data.

Mapping used: Female → 1 , Male → 0

This is called **Label Encoding (manual mapping)**.

```
print(df[['Sex_encoded', 'Survived']].corr())
```

Calculates correlation between:

- Encoded gender feature
- Survival outcome

## 2. DROPPING IRRELEVANT NUMERICAL FEATURES :

```
→ irrelevant_cols = ['PassengerId']
print("Dropping irrelevant columns :", irrelevant_cols)
df.drop(columns=irrelevant_cols,inplace=True,errors='ignore')
df.head()
```

**output :**

Dropping irrelevant columns : ['PassengerId']

|   | Survived | Pclass | Name                                              | Sex    | Age  | SibSp | Parch | Ticket           | Fare    | Cabin | Embarked | Sex_encoded |
|---|----------|--------|---------------------------------------------------|--------|------|-------|-------|------------------|---------|-------|----------|-------------|
| 0 | 0        | 3      | Braund, Mr. Owen Harris                           | male   | 22.0 | 1     | 0     | A/5 21171        | 7.2500  | NaN   | S        | 0           |
| 1 | 1        | 1      | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1     | 0     | PC 17599         | 71.2833 | C85   | C        | 1           |
| 2 | 1        | 3      | Heikinen, Miss. Laina                             | female | 26.0 | 0     | 0     | STON/O2. 3101282 | 7.9250  | NaN   | S        | 1           |
| 3 | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)      | female | 35.0 | 1     | 0     | 113803           | 53.1000 | C123  | S        | 1           |
| 4 | 0        | 3      | Allen, Mr. William Henry                          | male   | 35.0 | 0     | 0     | 373450           | 8.0500  | NaN   | S        | 0           |

Explanation :

```
irrelevant_cols = ['PassengerId']
```

You are marking **PassengerId** as an irrelevant feature because:

- It is just an ID number
- It does **not help** in predicting survival (no predictive power)

```
df.drop(columns=irrelevant_cols, inplace=True, errors='ignore')
```

This actually removes the column from dataset.

Meaning of parameters:

- **columns=irrelevant\_cols** → drop these columns
- **inplace=True** → modify original dataframe directly (no new copy)
- **errors='ignore'** → prevents error if column is already missing (safe coding)

### 3. DROPPING IRRELEVANT CATEOGORICAL FEATURES :

```
→ print(df['Name'].nunique())  
    print(df['Ticket'].nunique())  
    print(df['Cabin'].nunique())
```

output :

```
891  
681  
147
```

The `nunique()` function in pandas (defaulting to columns). It returns a Series with the number of unique values, excluding NaN values by default, making it ideal for identifying cardinalities in data analysis.

```
→ pd.crosstab(df['Sex'],df['Survived'],normalize='index')
```

output :

| Survived | 0        | 1        |
|----------|----------|----------|
| Sex      |          |          |
| female   | 0.257962 | 0.742038 |
| male     | 0.811092 | 0.188908 |

EDA using cross-tabulation shows that females had a significantly higher survival rate than males, indicating that the feature ‘Sex’ has strong predictive power and should be retained for modeling.”

```
→ pd.crosstab(df['Embarked'],df['Survived'])
```

output :

| Survived | 0   | 1   |
|----------|-----|-----|
| Embarked |     |     |
| C        | 75  | 93  |
| Q        | 47  | 30  |
| S        | 427 | 217 |

In the Titanic dataset, the Embarked feature represents the port of embarkation: C—Cherbourg, Q—Queenstown, and S—Southampton.

The port of embarkation is not random:

It is linked to ticket class

It correlates with socio-economic status

It indirectly affects survival probability

That's why Embarked is often a useful categorical feature

→ `irrelevant_cat_cols = ['Name', 'Ticket', 'Cabin']`

You are creating a list of categorical columns that you decided are not useful for prediction.

- **Name** → Almost every passenger has a unique name.  
→ High cardinality, no predictive pattern.
- **Ticket** → Mostly unique ticket numbers.  
→ Again high cardinality, not meaningful.
- **Cabin** → Many missing values (NaN).  
→ Too sparse, inconsistent, messy.

→ `df.drop(columns=irrelevant_cat_cols, inplace=True)`

- Removing those columns from the dataframe
- `inplace=True` → modifying the original dataframe directly (no copy created)

→ `df.head()`

output :

|   | Survived | Pclass | Sex    | Age  | SibSp | Parch | Fare    | Embarked | Sex_encoded |
|---|----------|--------|--------|------|-------|-------|---------|----------|-------------|
| 0 | 0        | 3      | male   | 22.0 | 1     | 0     | 7.2500  | S        | 0           |
| 1 | 1        | 1      | female | 38.0 | 1     | 0     | 71.2833 | C        | 1           |
| 2 | 1        | 3      | female | 26.0 | 0     | 0     | 7.9250  | S        | 1           |
| 3 | 1        | 1      | female | 35.0 | 1     | 0     | 53.1000 | S        | 1           |
| 4 | 0        | 3      | male   | 35.0 | 0     | 0     | 8.0500  | S        | 0           |

## 4.6 DATA PREPROCESSING ON TRAINING DATA

### 1. HANDLING MISSING VALUES :

```
→ print("Missing values in each column ")  
   print(df.isnull().sum())
```

#### output :

Missing values in each column

```
Survived    0  
Pclass      0  
Sex         0  
Age        177  
SibSp       0  
Parch       0  
Fare        0  
Embarked    2  
Sex_encoded  0  
dtype: int64
```

df.isnull() : Checks every cell in the dataframe. If a value is missing, it returns True. else returns False .

sum() : Counts the number of True values in each column.

### 2. HANDLING MISSING VALUE IN NUMERICAL COLUMN ( MEAN ) :

```
→ mean_value = df['Age'].mean()
```

Selects the Age column from the dataframe.

Computes the mean of all non missing values in that column.

Stores the result inside the variable mean\_value.

```
→ df['Age'] = df['Age'].fillna(mean_value)
```

Selects the Age column.

Finds all missing values in that column.

Replaces those missing values with mean\_value.

Updates the Age column with the new values.

```
→ print(df.isnull().sum())
```

#### output :

```
Survived    0  
Pclass      0  
Sex         0  
Age         0  
SibSp       0
```

```
Parch      0
Fare        0
Embarked    2
Sex_encoded  0
dtype: int64
```

### 3. HANDLING MISSING VALUE IN CATEGORICAL COLUMN ( MODE ) :

→ **mode\_value = df['Embarked'].mode()[0]**

Selects the Embarked column from the dataframe.  
Calculates the mode, which is the most common value.  
mode() returns a series of most frequent values.  
[0] selects the first mode value from that series.  
Stores it in the variable mode\_value.

→ **df['Embarked'] = df['Embarked'].fillna(mode\_value)**

Selects the Embarked column.  
Identifies missing values in that column.  
Replaces them with mode\_value.  
Updates the Embarked column with the new values.

→ **print(df.isnull().sum())**

**output :**

```
Survived      0
Pclass        0
Sex           0
Age           0
SibSp         0
Parch         0
Fare          0
Embarked      0
Sex_encoded    0
dtype: int64
```

### 4. ENCODING THE CATEGORICAL VARIABLES :

→ **df = df.drop(columns=['Sex'])**

Drop original categorical column 'Sex' (already encoded)

→ **df = pd.get\_dummies(df, columns=['Embarked'], drop\_first=True)**

Selects the Embarked column.  
Creates separate binary columns for each unique category.

Each new column contains 0 or 1 values.  
 1 indicates presence of that category.  
 0 indicates absence of that category.  
 Removes the original Embarked column.  
 Updates the dataframe with new encoded columns.

Meaning of drop\_first=True  
 Drops the first category column.  
 Prevents multicollinearity problem.  
 Avoids dummy variable trap in regression models.

Example  
 If Embarked has categories S, C, Q  
 New columns may be  
 Embarked\_C  
 Embarked\_Q

If a passenger boarded from S  
 Embarked\_C = 0  
 Embarked\_Q = 0

If from C  
 Embarked\_C = 1  
 Embarked\_Q = 0

Why It Is Used  
 Machine learning models require numerical input.  
 Categorical values cannot be directly used.  
 One Hot Encoding converts categories into model understandable format.

→ `df[['Embarked_Q', 'Embarked_S']] = df[['Embarked_Q', 'Embarked_S']].astype(int)`

Selects the columns Embarked\_Q and Embarked\_S.  
 Changes their data type to integer using astype(int).  
 Updates those columns in the dataframe.

Why It Is Needed ?  
 After One Hot Encoding, values may sometimes be in boolean type True or False.  
 Machine learning models generally expect numerical values like 0 and 1.  
 Converting to integer ensures proper numeric format.

→ `df.head()`

|   | Survived | Pclass | Age  | SibSp | Parch | Fare    | Sex_encoded | Embarked_Q | Embarked_S |
|---|----------|--------|------|-------|-------|---------|-------------|------------|------------|
| 0 | 0        | 3      | 22.0 | 1     | 0     | 7.2500  | 0           | 0          | 1          |
| 1 | 1        | 1      | 38.0 | 1     | 0     | 71.2833 | 1           | 0          | 0          |
| 2 | 1        | 3      | 26.0 | 0     | 0     | 7.9250  | 1           | 0          | 1          |
| 3 | 1        | 1      | 35.0 | 1     | 0     | 53.1000 | 1           | 0          | 1          |
| 4 | 0        | 3      | 35.0 | 0     | 0     | 8.0500  | 0           | 0          | 1          |

## 5. FEATURE SCALING :

→ **scaling\_features = ['Age','Fare']**

Stores the column names Age and Fare inside a list.

This list is assigned to the variable `scaling_features`.

Used later for applying scaling methods like Standardization or Normalization.

Why It Is Used

Age and Fare may have different value ranges.

Large differences in scale can affect machine learning models.

Scaling ensures both features contribute fairly during training.

→ **scaler = StandardScaler()**

Initializes the `StandardScaler` class.

Stores the scaler object inside the variable `scaler`.

This object will later be used to scale numerical features.

Transforms data so that

Mean becomes 0 and Standard deviation becomes 1

Scaled value =  $(\text{Value} - \text{Mean}) / \text{Standard Deviation}$

Why It Is Used

Ensures all numerical features are on the same scale.

Improves performance of models like Logistic Regression, SVM, KNN.

Prevents features with large values from dominating smaller ones.

→ **df[scaling\_features] = scaler.fit\_transform(df[scaling\_features])**

Selects the columns stored in `scaling_features`.

Fits the `StandardScaler` on those columns.

Calculates mean and standard deviation for each column.

Transforms the values using standardization formula.

Replaces original values with scaled values in the dataframe.

Fit : Calculates mean and standard deviation from the data.

Transform :  $\text{New value} = (\text{Value} - \text{Mean}) / \text{Standard Deviation}$ .

Mean of each selected column becomes 0 and Standard deviation becomes 1.

Values may become negative or decimal

→ **df.head ()**

|   | Survived | Pclass | Age       | SibSp | Parch | Fare      | Sex_encoded | Embarked_Q | Embarked_S |
|---|----------|--------|-----------|-------|-------|-----------|-------------|------------|------------|
| 0 | 0        | 3      | -0.592481 | 1     | 0     | -0.502445 | 0           | 0          | 1          |
| 1 | 1        | 1      | 0.638789  | 1     | 0     | 0.786845  | 1           | 0          | 0          |
| 2 | 1        | 3      | -0.284663 | 0     | 0     | -0.488854 | 1           | 0          | 1          |
| 3 | 1        | 1      | 0.407926  | 1     | 0     | 0.420730  | 1           | 0          | 1          |
| 4 | 0        | 3      | 0.407926  | 0     | 0     | -0.486337 | 0           | 0          | 1          |



## 4.7 FEATURE SELECTION -DATA PREPARATION

### 1. FINAL FEATURE MATRIX

```
selected_features = ['Pclass',  
'Age','Fare','Sex_encoded','Embarked_Q','Embarked_S']
```

```
X = df[selected_features]
```

```
Y = df['Survived']
```

```
→ X.head()
```

|   | Pclass | Age       | Fare      | Sex_encoded | Embarked_Q | Embarked_S |
|---|--------|-----------|-----------|-------------|------------|------------|
| 0 | 3      | -0.592481 | -0.502445 | 0           | 0          | 1          |
| 1 | 1      | 0.638789  | 0.786845  | 1           | 0          | 0          |
| 2 | 3      | -0.284663 | -0.488854 | 1           | 0          | 1          |
| 3 | 1      | 0.407926  | 0.420730  | 1           | 0          | 1          |
| 4 | 3      | 0.407926  | -0.486337 | 0           | 0          | 1          |

```
→ Y.head()
```

```
0    0  
1    1  
2    1  
3    1  
4    0  
Name: Survived, dtype: int64
```

## 4.8 MODEL BUILDING

### ALGORITHMS :

#### 1. LOGISTIC REGRESSION

#### 2. DECISION TREE

#### 3. RANDOM FOREST

##### 1. Logistic Regression

###### Description

Supervised machine learning algorithm used for classification problems.

Predicts probability of classes using a sigmoid function.

###### Working Principle

Uses linear equation and converts output into probability between 0 and 1.

If probability is greater than threshold (usually 0.5), class is predicted as positive.

###### Advantages

Simple and easy to implement.

Works well for binary classification.

Disadvantages

Not suitable for complex non linear data.

Use Cases

Spam detection.

Medical diagnosis classification.

## **2 .Decision Tree**

Description

Supervised machine learning algorithm that makes decisions using tree like structure.

Working Principle

Splits data based on feature conditions.

Each internal node represents a decision.

Leaf nodes represent final output class.

Advantages

Easy to understand and interpret.

Handles both numerical and categorical data.

Disadvantages

Prone to overfitting.

Use Cases

Customer segmentation.

Credit risk analysis.

## **3. Random Forest**

Description

Ensemble machine learning algorithm that combines multiple decision trees.

Working Principle

Creates multiple decision trees using random samples.

Combines results using voting (classification) or averaging (regression).

Advantages

Reduces overfitting compared to single decision tree.

Provides higher accuracy.

Disadvantages

More computationally expensive.

Less interpretable compared to single tree.

Use Cases

Fraud detection.

Image classification.

## **2. SPLIT THE TRAIN DATA INTO TRAIN AND VALIDATION DATA**

→ `X_train , X_validate ,Y_train , Y_validate = train_test_split(X,Y,test_size=0.2,random_state = 42,stratify = Y)`

Splits input features (X) and target labels (Y) into two parts.  
Training data is used to train the model.  
Validation data is used to test model performance.

#### Parameters Explanation

`test_size = 0.2`

20 percent of data is used for validation.

80 percent of data is used for training.

`random_state = 42`

Ensures reproducibility of results.

Same split will occur every time the code is run.

`stratify = Y`

Maintains same class distribution in training and validation sets.

Important for imbalanced datasets.

#### Output Variables

`X_train` and `Y_train`

Used for model training.

`X_validate` and `Y_validate`

Used for model evaluation.

### 3. TRAIN MODEL ON TRAIN SPLIT USING 3 ALGORITHMS

→ `lgr_model = LogisticRegression(max_iter=1000)`  
`lgr_model.fit(X_train, Y_train)`

Logistic Regression is a supervised machine learning classification algorithm based on statistical probability.

Works using the sigmoid function.

Model first calculates linear combination of input features using formula

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Where

$w$  = weights

$x$  = input features

$b$  = bias

Then sigmoid function is applied

$$\text{Sigmoid}(z) = 1 / (1 + e^{-z})$$

This converts output into probability between 0 and 1.

If probability > 0.5, class is predicted as 1 else 0.

`fit()`

uses optimization algorithms like gradient descent or similar solvers.

Adjusts weights and bias to minimize loss function (log loss / cross entropy loss).

`max_iter=1000` allows more iterations for better convergence.

## Purpose

Used for classification problems where probability based prediction is needed.

---

```
→ dt_model = DecisionTreeClassifier(criterion='gini', max_depth=None,  
random_state=42)  
dt_model.fit(X_train, Y_train)
```

Decision Tree is a rule based supervised classification algorithm that uses tree structure decisions.

Works by recursively splitting data based on feature thresholds.

At each node, algorithm selects best feature to split using impurity measures.

criterion='gini'

Uses Gini impurity to measure node purity.

$Gini = 1 - \sum (p_i^2)$

Where  $p_i$  is probability of each class.

Lower Gini value means better split.

## Tree Building Process

Select best feature.

Split dataset into subsets.

Repeat process until stopping condition is reached.

## Stopping Conditions

Pure node achieved.

Maximum depth reached.

Minimum samples condition reached.

random\_state=42 ensures reproducibility.

## Purpose

Used when interpretability of model decisions is important.

---

```
→ rf_model = RandomForestClassifier(n_estimators=100, random_state=42)  
rf_model.fit(X_train, Y_train)
```

Random Forest is an ensemble learning algorithm that combines multiple decision trees.

## Internal Working Core Logic

Works using bagging (Bootstrap Aggregating).

## Process

Creates multiple random subsets of training data using bootstrapping.

Each subset trains one decision tree independently.

Each tree makes its own prediction.

## Final Prediction

Classification

Majority voting is used.

Regression (if used)

Average of all tree outputs is taken.

`n_estimators=100`

Creates 100 decision trees.

More trees generally improve accuracy but increase computation cost.

`random_state=42` ensures consistent random sampling.

Advantages

Reduces overfitting compared to single decision tree.

Improves prediction stability

#### **4. PREDICT ON VALIDATION DATA**

→ `Y_validate_predict_lgr = lgr_model.predict(X_validate)`

→ `Y_validate_predict_dt = dt_model.predict(X_validate)`

→ `Y_validate_predict_rf = rf_model.predict(X_validate)`

Each model uses its learned patterns from training data.

Then predicts class labels for validation dataset features.

Internal Working Logic

Logistic Regression Prediction

Calculates probability using sigmoid function.

If probability is greater than threshold (usually 0.5), class label is assigned.

Decision Tree Prediction

Passes validation data through decision rules created during training.

Follows tree path based on feature conditions until reaching leaf node.

Leaf node gives final class prediction.

Random Forest Prediction

Each tree inside forest makes its own prediction.

Final prediction is decided using majority voting from all trees.

Output Variables

`Y_validate_predict_lgr`

Stores predictions from Logistic Regression model.

`Y_validate_predict_dt`

Stores predictions from Decision Tree model.

`Y_validate_predict_rf`

Stores predictions from Random Forest model.

#### **4.9 EVALUATION METRICS ( USING VALIDATION DATA )**

##### **1.ACCURACY :**

`accuracy = accuracy_score(Y_validate ,Y_validate_predict_lgr)`

```
print ( f"{'VALIDATION ACCURACY USING LOGISTIC REGRESSION':  
<50}:{accuracy}")
```

```
accuracy = accuracy_score(Y_validate,Y_validate_predict_dt)  
print (f"{'VALIDATION ACCURACY USING DECISION TREE':  
<50}:{accuracy}")
```

```
accuracy = accuracy_score(Y_validate,Y_validate_predict_rf)  
print (f"{'VALIDATION ACCURACY USING RANDOM FOREST  
CLASSIFIER': <50}:{accuracy}")
```

output:

```
VALIDATION ACCURACY USING LOGISTIC REGRESSION      :0.776536312849162  
VALIDATION ACCURACY USING DECISION TREE            :0.7932960893854749  
VALIDATION ACCURACY USING RANDOM FOREST CLASSIFIER:0.8100558659217877
```

## 2. CONFUSION MATRIX

```
cm = confusion_matrix(Y_validate,Y_validate_predict_lgr)  
print ('CONFUSION MATRIX USING LOGISTIC REGRESSION \n',cm)
```

```
cm = confusion_matrix(Y_validate,Y_validate_predict_dt)  
print ('CONFUSION MATRIX USING DECISION TREES\n',cm)
```

```
cm = confusion_matrix(Y_validate,Y_validate_predict_rf)  
print ('CONFUSION MATRIX USING RANDOM FOREST CLASSIFIER  
\n',cm)
```

output:

```
CONFUSION MATRIX USING LOGISTIC REGRESSION  
[[94 16]  
 [24 45]]  
CONFUSION MATRIX USING DECISION TREES  
[[93 17]  
 [20 49]]  
CONFUSION MATRIX USING RANDOM FOREST CLASSIFIER  
[[96 14]  
 [20 49]]
```

## 3.PRECISION

```
precision = precision_score(Y_validate,Y_validate_predict_lgr)  
print (f"{'PRECISION USING LOGISTIC REGRESSION': <50}:{precision}")
```

```
precision = precision_score(Y_validate,Y_validate_predict_dt)  
print (f"{'PRECISION USING DECISION TREE': <50}:{precision}")
```

```
precision = precision_score(Y_validate,Y_validate_predict_rf)
print (f'{'PRECISION USING RANDOM FOREST CLASSIFIER':<50}:{precision}')
```

output:

```
PRECISION USING LOGISTIC REGRESSION      :0.7377049180327869
PRECISION USING DECISION TREE             :0.7424242424242424
PRECISION USING RANDOM FOREST CLASSIFIER  :0.7777777777777778
```

#### 4. RECALL

```
recall = recall_score(Y_validate,Y_validate_predict_lgr)
print (f'{"RECALL USING LOGISTIC REGRESSION":<50}:{recall}')
```

```
recall = recall_score(Y_validate,Y_validate_predict_dt)
print (f'{"RECALL USING DECISION TREE": <50}:{recall}')
```

```
recall = recall_score(Y_validate,Y_validate_predict_rf)
print (f'{"RECALL USING RANDOM FOREST CLASSIFIER":<50}:{recall}')
```

output:

```
RECALL USING LOGISTIC REGRESSION          :0.6521739130434783
RECALL USING DECISION TREE               :0.7101449275362319
RECALL USING RANDOM FOREST CLASSIFIER    :0.7101449275362319
```

#### 5. F1-SCORE:

```
f1 = f1_score(Y_validate,Y_validate_predict_lgr)
print(f'{'F1 SCORE USING LOGISTIC REGRESSION':<50}:{f1}')
```

```
f1 = f1_score(Y_validate,Y_validate_predict_dt)
print(f'{'F1 SCORE USING DECISION TREE':<50}:{f1}')
```

```
f1 = f1_score(Y_validate,Y_validate_predict_rf)
print(f'{'F1 SCORE USING RANDOM FOREST CLASSIFIER':<50}:{f1}')
```

output:

```
F1 SCORE USING LOGISTIC REGRESSION        :0.6923076923076923
F1 SCORE USING DECISION TREE              :0.725925925925926
F1 SCORE USING RANDOM FOREST CLASSIFIER   :0.7424242424242424
```

#### 6. CLASSIFICATION REPORT:

```
print('LOGISTIC REGRESSION :\n')
print (classification_report(Y_validate,Y_validate_predict_lgr))
```

```

print("\nDECISION TREE :\n")
print(classification_report(Y_validate,Y_validate_predict_dt))

print("\nRANDOM FOREST CLASSIFIER :")
print(classification_report(Y_validate,Y_validate_predict_rf))

```

output:

| LOGISTIC REGRESSION : |           |        |          |         |  |
|-----------------------|-----------|--------|----------|---------|--|
|                       | precision | recall | f1-score | support |  |
| 0                     | 0.80      | 0.85   | 0.82     | 110     |  |
| 1                     | 0.74      | 0.65   | 0.69     | 69      |  |
| accuracy              |           |        | 0.78     | 179     |  |
| macro avg             | 0.77      | 0.75   | 0.76     | 179     |  |
| weighted avg          | 0.77      | 0.78   | 0.77     | 179     |  |

| DECISION TREE : |           |        |          |         |  |
|-----------------|-----------|--------|----------|---------|--|
|                 | precision | recall | f1-score | support |  |
| 0               | 0.82      | 0.85   | 0.83     | 110     |  |
| 1               | 0.74      | 0.71   | 0.73     | 69      |  |
| accuracy        |           |        | 0.79     | 179     |  |
| macro avg       | 0.78      | 0.78   | 0.78     | 179     |  |
| weighted avg    | 0.79      | 0.79   | 0.79     | 179     |  |

| RANDOM FOREST CLASSIFIER : |           |        |          |         |  |
|----------------------------|-----------|--------|----------|---------|--|
|                            | precision | recall | f1-score | support |  |
| 0                          | 0.83      | 0.87   | 0.85     | 110     |  |
| 1                          | 0.78      | 0.71   | 0.74     | 69      |  |
| accuracy                   |           |        | 0.81     | 179     |  |
| macro avg                  | 0.80      | 0.79   | 0.80     | 179     |  |
| weighted avg               | 0.81      | 0.81   | 0.81     | 179     |  |

## 4.10 CHOOSING BEST MODEL

### 1. FITTING RANDOM FOREST ON FULL TRAINING DATA

→ `rf_model.fit(X,Y)`

output:



| RandomForestClassifier ⓘ ?                        |  |        |
|---------------------------------------------------|--|--------|
| Parameters                                        |  |        |
| <input type="checkbox"/> n_estimators             |  | 100    |
| <input type="checkbox"/> criterion                |  | 'gini' |
| <input type="checkbox"/> max_depth                |  | None   |
| <input type="checkbox"/> min_samples_split        |  | 2      |
| <input type="checkbox"/> min_samples_leaf         |  | 1      |
| <input type="checkbox"/> min_weight_fraction_leaf |  | 0.0    |
| <input type="checkbox"/> max_features             |  | 'sqrt' |
| <input type="checkbox"/> max_leaf_nodes           |  | None   |
| <input type="checkbox"/> min_impurity_decrease    |  | 0.0    |
| <input type="checkbox"/> bootstrap                |  | True   |
| <input type="checkbox"/> oob_score                |  | False  |
| <input type="checkbox"/> n_jobs                   |  | None   |
| <input type="checkbox"/> random_state             |  | 42     |
| <input type="checkbox"/> verbose                  |  | 0      |
| <input type="checkbox"/> warm_start               |  | False  |
| <input type="checkbox"/> class_weight             |  | None   |
| <input type="checkbox"/> ccp_alpha                |  | 0.0    |
| <input type="checkbox"/> max_samples              |  | None   |
| <input type="checkbox"/> monotonic_cst            |  | None   |

#### 4.11 LOADING TEST DATASET:

→ `df_test = pd.read_csv("./titanic/test.csv")`

#### 4.12 VERIFYING THE LOADED TEST DATASET

##### 1. INSPECTING THE COLUMN NAMES

→ `df_test.columns`

output:

```
Index(['PassengerId', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch',
      'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='str')
```

## 2. INSPECTING TOTAL SAMPLE RECORDS

→ `df_test.head()`

|   | PassengerId | Pclass | Name                                         | Sex    | Age  | SibSp | Parch | Ticket  | Fare    | Cabin | Embarked |
|---|-------------|--------|----------------------------------------------|--------|------|-------|-------|---------|---------|-------|----------|
| 0 | 892         | 3      | Kelly, Mr. James                             | male   | 34.5 | 0     | 0     | 330911  | 7.8292  | NaN   | Q        |
| 1 | 893         | 3      | Wilkes, Mrs. James (Ellen Needs)             | female | 47.0 | 1     | 0     | 363272  | 7.0000  | NaN   | S        |
| 2 | 894         | 2      | Myles, Mr. Thomas Francis                    | male   | 62.0 | 0     | 0     | 240276  | 9.6875  | NaN   | Q        |
| 3 | 895         | 3      | Wirz, Mr. Albert                             | male   | 27.0 | 0     | 0     | 315154  | 8.6625  | NaN   | S        |
| 4 | 896         | 3      | Hirvonen, Mrs. Alexander (Helga E Lindqvist) | female | 22.0 | 1     | 1     | 3101298 | 12.2875 | NaN   | S        |

## 3. INSPECTING TOTAL NUMBER OF RECORDS AND COLUMNS

→ `print(len(df_test))`

438

→ `print(len(df_test.columns))`

11

## 4.13 DATA PREPROCESSING ON TEST DATA

### 1. HANDLING THE MISSING VALUES

→ `print("Missing values in each column ")`

→ `print(df_test.isnull().sum())`

Missing values in each column

PassengerId 0

Pclass 0

Name 0

Sex 0

Age 86

SibSp 0

Parch 0

Ticket 0

Fare 1

Cabin 327

Embarked 0

dtype: int64

### 2. HANDLING MISSING VALUES IN NUMERICAL COLUMN ( MEAN AND MEDIAN )

→ `df_test['Age']=df_test['Age'].fillna( mean_value)`

→ `df_test.isnull().sum()`

output :

PassengerId 0

Pclass 0

Name 0

```
Sex      0
Age      0
SibSp    0
Parch    0
Ticket   0
Fare     1
Cabin    327
Embarked  0
dtype: int64
```

```
→ median_value = df_test['Fare'].median()
```

```
→ df_test['Fare'] = df_test['Fare'].fillna(median_value)
```

```
→ df_test.isnull().sum()
```

**output :**

```
PassengerId  0
Pclass       0
Name         0
Sex          0
Age          0
SibSp        0
Parch        0
Ticket       0
Fare         0
Cabin        327
Embarked     0
dtype: int64
```

### 3. REMOVING THE IRRELEVANT OR REDUNDANT FEATURES

```
→ irrelevant_cols_test = ['PassengerId','Name','Ticket','Cabin']
```

```
→ print("Dropping irrelevant columns :", irrelevant_cols_test)
```

**output:**

```
Dropping irrelevant columns : ['PassengerId', 'Name', 'Ticket', 'Cabin']
```

```
→ df_test.drop(columns=irrelevant_cols_test,inplace=True,errors='ignore')
```

```
→ df_test.head()
```

**output:**

|   | Pclass | Sex    | Age  | SibSp | Parch | Fare    | Embarked |
|---|--------|--------|------|-------|-------|---------|----------|
| 0 | 3      | male   | 34.5 | 0     | 0     | 7.8292  | Q        |
| 1 | 3      | female | 47.0 | 1     | 0     | 7.0000  | S        |
| 2 | 2      | male   | 62.0 | 0     | 0     | 9.6875  | Q        |
| 3 | 3      | male   | 27.0 | 0     | 0     | 8.6625  | S        |
| 4 | 3      | female | 22.0 | 1     | 1     | 12.2875 | S        |

#### 4. ENCODING THE CATEGORICAL VARIABLES

→ `df_test['Sex_encoded'] = df_test['Sex'].map({'female': 1, 'male': 0})`

→ `df_test= df_test.drop(columns='Sex')`

→ `df_test = pd.get_dummies(df_test,columns=['Embarked'],drop_first=True)`

→ `df_test[['Embarked_Q','Embarked_S']]`  
`=df_test[['Embarked_Q','Embarked_S']].astype(int)`

→ `df_test.head()`

output:

|   | Pclass | Age  | SibSp | Parch | Fare    | Sex_encoded | Embarked_Q | Embarked_S |
|---|--------|------|-------|-------|---------|-------------|------------|------------|
| 0 | 3      | 34.5 | 0     | 0     | 7.8292  | 0           | 1          | 0          |
| 1 | 3      | 47.0 | 1     | 0     | 7.0000  | 1           | 0          | 1          |
| 2 | 2      | 62.0 | 0     | 0     | 9.6875  | 0           | 1          | 0          |
| 3 | 3      | 27.0 | 0     | 0     | 8.6625  | 0           | 0          | 1          |
| 4 | 3      | 22.0 | 1     | 1     | 12.2875 | 1           | 0          | 1          |

#### 5. FEATURE SCALING:

→ `df_test[['Age', 'Fare']] = scaler.transform(df_test[['Age', 'Fare']])`

→ `df_test.head()`

output:

|   | Pclass | Age       | SibSp | Parch | Fare      | Sex_encoded | Embarked_Q | Embarked_S |
|---|--------|-----------|-------|-------|-----------|-------------|------------|------------|
| 0 | 3      | 0.369449  | 0     | 0     | -0.490783 | 0           | 1          | 0          |
| 1 | 3      | 1.331378  | 1     | 0     | -0.507479 | 1           | 0          | 1          |
| 2 | 2      | 2.485693  | 0     | 0     | -0.453367 | 0           | 1          | 0          |
| 3 | 3      | -0.207709 | 0     | 0     | -0.474005 | 0           | 0          | 1          |
| 4 | 3      | -0.592481 | 1     | 1     | -0.401017 | 1           | 0          | 1          |

## 4.14 FEATURE SELECTION

```
→ selected_features = [  
    'Pclass',  
    'Age',  
    'Fare',  
    'Sex_encoded',  
    'Embarked_Q',  
    'Embarked_S'  
]  
  
→ X_test = df_test[selected_features]  
→ X_test.head()
```

|   | Pclass | Age       | Fare      | Sex_encoded | Embarked_Q | Embarked_S |
|---|--------|-----------|-----------|-------------|------------|------------|
| 0 | 3      | 0.369449  | -0.490783 | 0           | 1          | 0          |
| 1 | 3      | 1.331378  | -0.507479 | 1           | 0          | 1          |
| 2 | 2      | 2.485693  | -0.453367 | 0           | 1          | 0          |
| 3 | 3      | -0.207709 | -0.474005 | 0           | 0          | 1          |
| 4 | 3      | -0.592481 | -0.401017 | 1           | 0          | 1          |

### 1. SANITY CHECK:

```
→ X.shape  
(891, 6)  
  
→ X_test.shape  
(418, 6)
```

## 4.15 PREDICTION:

```
→ Y_test = rf_model.predict(X_test)  
→ Y_test = pd.Series(Y_test)  
→ Y_test.head()
```

output:

```
0  0  
1  0  
2  0  
3  1  
4  0  
dtype: int64
```

## 4.16 TEST WITH CUSTOM INPUT

```
def predict_survival_interactive(rf_model, selected_features):
    print("Enter passenger details to predict survival:")

    # Input prompts
    pclass = int(input("Pclass (1 = 1st, 2 = 2nd, 3 = 3rd): "))
    age = float(input("Age (in years): "))
    fare = float(input("Fare (ticket price): "))
    sex = input("Sex (male/female): ").strip().lower()
    embarked = input("Embarked (C/Q/S): ").strip().upper()

    # Encode features exactly matching dataset column names
    sex_encoded = 1 if sex == 'female' else 0
    embarked_Q = 1 if embarked == 'Q' else 0
    embarked_S = 1 if embarked == 'S' else 0

    # Create DataFrame with same column order as selected_features
    input_df = pd.DataFrame([
        pclass,
        age,
        fare,
        sex_encoded,
        embarked_Q,
        embarked_S
    ], columns=selected_features)

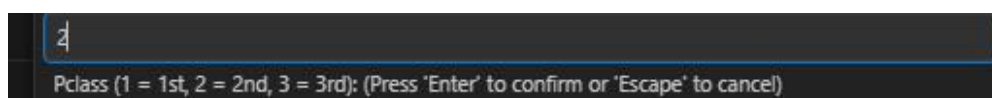
    # Make prediction
    prediction = rf_model.predict(input_df)[0]

    # Output result
    if prediction == 1:
        print("\nResult: Passenger is likely to SURVIVE")
    else:
        print("\nResult: Passenger is likely to NOT SURVIVE")
# Make sure rf_model is already trained on full training data
# selected_features =
['Pclass', 'Age', 'Fare', 'Sex_encoded', 'Embarked_Q', 'Embarked_S']

predict_survival_interactive(rf_model, selected_features)
```

### OUTPUT:

Enter passenger details to predict survival:



16

Age (in years): (Press 'Enter' to confirm or 'Escape' to cancel)

48976

Fare (ticket price): (Press 'Enter' to confirm or 'Escape' to cancel)

male

Sex (male/female): (Press 'Enter' to confirm or 'Escape' to cancel)

C

Embarked (C/Q/S): (Press 'Enter' to confirm or 'Escape' to cancel)

**Result: Passenger is likely to NOT SURVIVE**